

# Addressing Structural Distribution Shift in Explanations for Graph Neural Networks

Zhuomin Chen<sup>1</sup>, Hojat Allah Salehi<sup>1</sup>, Esteban Schafir<sup>1</sup>, Xu Zheng<sup>1</sup>, Jiaxing Zhang<sup>1</sup>, Hua Wei<sup>2</sup>,  
Jingchao Ni<sup>3</sup>, Farhad Shirani<sup>1</sup>, and Dongsheng Luo<sup>1</sup>, *Member, IEEE*

**Abstract**—Graph Neural Networks (GNNs) are essential for processing graph-structured data and have wide applications in critical domains. The increasing use of GNNs in high-stakes scenarios requires robust explainability to ensure trust and transparency in decision-making. A common approach to explaining GNNs is to identify subgraphs, *a.k.a.* explanations, that significantly influence model predictions. However, this task is challenging due to the distribution shifts from the original training graphs to the explanation subgraphs, a factor that is largely overlooked in the existing research. These shifts arise because GNNs are trained on original graphs, while explanation subgraphs often differ in properties such as the number of nodes or structural patterns. As a result, GNNs may struggle to generalize to explanation subgraphs with a different distribution from its training data. In this paper, we systematically investigate the Out-Of-Distribution (OOD) problem through theoretical analysis and empirical studies. To address this challenge, we first develop a theoretical framework that formalizes the notion of explanation subgraphs through *sufficiency* and *minimality* criteria, ensuring both prediction preservation and structural compactness. Our analysis reveals a fundamental distributional disparity between explanation subgraphs and original graphs, leading to a novel concept of *proxy graphs* proposed in this work. Proxy graphs maintain the essential explanatory information while conforming to the original data distribution through a combination of parametric and non-parametric optimization approaches. Empirical evaluations on diverse datasets show that our method improves the quality and reliability of GNN explanations, advancing the field of GNN explainability.

**Index Terms**—Explainable artificial intelligence, graph neural networks, deep learning.

Received 15 February 2025; revised 1 April 2026; accepted 25 April 2026. Date of publication 4 May 2026; date of current version 6 August 2026. This work was supported in part by NSF under Grant IIS-2529283 and Grant ECCS-2242700. Recommended for acceptance by X. Cao. (*Corresponding author: Dongsheng Luo.*)

Zhuomin Chen, Hojat Allah Salehi, Esteban Schafir, and Xu Zheng are with Florida International University, Miami, FL 33199 USA (e-mail: zchen051@fiu.edu; hsalehi@fiu.edu; escha032@fiu.edu; xzhen019@fiu.edu).

Jiaxing Zhang was with New Jersey Institute of Technology, Newark, NJ 07102 USA. He is now with the TikTok Inc., Bellevue, WA 98004 USA (e-mail: tabzhangjx@gmail.com).

Hua Wei is with Arizona State University, Tempe, AZ 85287 USA (e-mail: hua.wei@asu.edu).

Jingchao Ni is with the University of Houston, Houston, TX 77004 USA (e-mail: jni7@uh.edu).

Farhad Shirani was with Florida International University, Miami, FL 33199 USA (e-mail: fshirani@fiu.edu).

Dongsheng Luo was with Florida International University, Miami, FL 33199 USA. He is now with Singapore Management University, Singapore 188065 (e-mail: luodongsheng01@gmail.com).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TPAMI.2026.3690304>, provided by the authors.

Digital Object Identifier 10.1109/TPAMI.2026.3690304

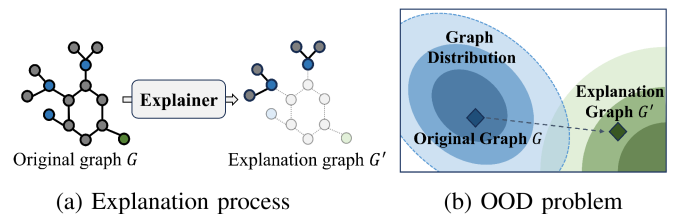


Fig. 1. Examples of the explanation process and the out-of-distribution problem. (a) is the explanation process for a graph learning model. The original graph  $G$  undergoes an explanation process, resulting in an explanation graph  $G_{exp}$  that highlights the most significant features and relationships; (b) shows that the explanation graph  $G_{exp}$  is out of distribution where the GNN is trained.

## I. INTRODUCTION

GRAPH Neural Networks (GNNs) have emerged as a pivotal technology for handling graph-structured data, demonstrating remarkable performance in various applications, including node classification and link prediction [18], [23], [37], [41]. Their growing use in critical sectors such as fraud detection and healthcare has escalated the need for explainability in their decision-making processes [26], [44], [49]. To meet this demand, a variety of explanation methods have recently been developed to interpret the behavior of GNN models [48].

A prominent approach to explaining GNNs is to extract meaningful subgraphs from input graphs that align with the models' predictions [28], [38], [40], [43], [47]. For example, in social networks, extracting communities or key connections can reveal influential relationships that drive network behaviors [34]; in molecular property prediction tasks, identifying substructures such as functional groups or molecular motifs helps researchers understand how specific chemical components contribute to a compound's activity [45]. To achieve this, various methods have been proposed, each offers a unique perspective. Among them, perturbation-based methods [28], [47] selectively remove nodes, edges, or subgraphs from a graph, and then measure the change in model predictions, whereby determine their impacts. These methods have achieved state-of-the-art performance in a wide range of benchmark datasets and applications.

Although it is intuitively correct, the underlying assumption of the aforementioned approach – GNNs can make accurate predictions on explanation subgraphs – may not always hold. As shown in Fig. 1, explanation subgraphs can significantly deviate from the structural distribution of original graphs, leading to an Out-of-Distribution (OOD) problem [15], [17], [51], [53]. For instance, in the widely used MUTAG dataset [13], each

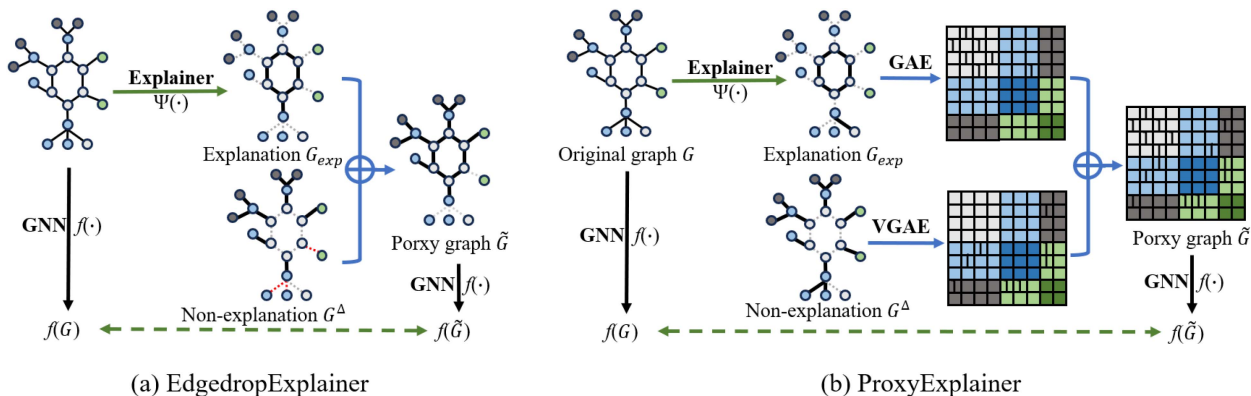


Fig. 2. Illustrations of (a) EdgedropExplainer, which applies the explainer to obtain  $G_{exp}$ , then constructs the complement subgraph  $G - G_{exp}$  and samples its edges with probability  $1 - p$  to generate  $G^\Delta$ . The final proxy graph  $\tilde{G}$  is formed by combining  $G_{exp}$  and  $G^\Delta$ . (b) ProxyExplainer, which consists of two components: an Explainer and a Proxy Graph Generator. The Explainer takes a graph  $G$  as input and produces an explanation subgraph  $G_{exp}$ , the proxy generator creates in-distribution proxy graphs that preserve the label information in  $G_{exp}$ . The proxy generator consists of a Graph Autoencoder (GAE) and a Variational Graph Autoencoder (VGAE).

TABLE I  
STATISTICS OF MOLECULE DATASETS FOR GRAPH CLASSIFICATION WITH GROUND-TRUTH EXPLANATIONS

|               | MUTAG | Benzene | Alkane-Carbonyl | Fluoride-Carbonyl | BA-2motifs | BA-3motifs | D&D    | OGBG-MolHIV | IMDB-BINARY | REDDIT-BINARY |
|---------------|-------|---------|-----------------|-------------------|------------|------------|--------|-------------|-------------|---------------|
| Graphs        | 4,337 | 12,000  | 4,326           | 8,671             | 1,000      | 3,000      | 1,178  | 41,127      | 1,000       | 2,000         |
| Average nodes | 29.15 | 20.58   | 21.13           | 21.36             | 25.00      | 21.92      | 284.32 | 25.51       | 19.77       | 429.63        |
| Average edges | 60.83 | 43.65   | 44.95           | 45.37             | 25.48      | 29.51      | 715.66 | 54.93       | 96.53       | 497.75        |
| Node features | 14    | 14      | 14              | 14                | 10         | 4          | 89     | 9           | 1           | 1             |
| GCN accuracy  | 0.80  | 0.92    | 0.98            | 0.95              | 1.00       | 0.98       | 0.80   | 0.97        | 0.71        | 0.84          |

graph represents a molecule, with nodes symbolizing atoms and edges indicating chemical bonds. The molecular graphs in this dataset usually contain hundreds of edges. In contrast, the  $NO_2$  functional group – a key subgraph influencing positive mutagenicity in a molecule – comprises merely 2 edges. This stark contrast in structures leads to a significant difference in the structural distributions of explanation subgraphs and original graphs. Since the model was trained with original graphs, the reliability of predictions on small size subgraphs is undermined due to the distribution shift problem.

Several pioneering studies have attempted to address this distributional challenge [17], [50], [51]. For example, CGE regards the GNN model as a transparent, fully accessible system, treating it as a “teacher” and training a “student” to predict labels for explanation subgraphs [17]. MixupExplainer [51] takes a first step toward addressing this distribution shift by combining the explanation subgraph with a non-explanation subgraph sampled from a different input graph. While effective in certain settings, it has notable limitations. First, the structural distribution shift problem affects perturbation-based GNN explainers broadly [28], [47], yet MixupExplainer addresses it as a remedy within a specific GIB-based formulation. This leaves the problem without a solid theoretical foundation that generalizes across different explainer designs and evaluation frameworks. Second, MixupExplainer constructs mixed graphs by unconditionally sampling a non-explanation subgraph from a different input graph, implicitly assuming independence between the explanation and non-explanation structures. This assumption is often violated in practice. For instance, in molecular graphs [21], [39],

once a functional group is fixed as the explanation, the remaining scaffold is constrained by valence rules and ring closure patterns, so the resulting mixed graph may itself lie outside the training distribution. These limitations motivate a principled framework that formally analyzes explanation-induced distribution shift and provides theoretically grounded solutions.

In this work, we systematically investigate the structural OOD problem in GNN explainability with both theoretical analysis and empirical studies. Our theoretical contribution starts by introducing a rigorous framework that formulates graph explainability through two fundamental information-theoretic measures: the *minimality* measure, which quantifies the conciseness of explanations, and the *sufficiency* measure, which captures their informativeness. We demonstrate that the popular training objectives of explainers, including the graph information bottleneck loss function, can be interpreted as approximations of these measures. Meanwhile, we identify a critical limitation: these approximations become unreliable when the distribution of subgraph explanations diverges from that of the original graphs, which may be caused by the differences in the size of the original graphs and their explanations. To address these fundamental challenges, we propose a novel concept – *proxy graph*, which serves as an intermediary between an explanation subgraph and its original graph. We develop two complementary approaches to construct these proxy graphs. Our non-parametric method generates proxy graphs through principled edge manipulation operations, providing a straightforward solution with theoretical guarantees on distribution preservation. To handle more complex graph structures, we further introduce a parametric approach

TABLE II  
COMPLEXITY COMPARISON OF EXPLANATION METHODS

| Method            | Time Complexity          | Space Complexity       |
|-------------------|--------------------------|------------------------|
| PGExplainer       | $O( \mathcal{E} )$       | $O( \mathcal{E}  + n)$ |
| MixupExplainer    | $O( \mathcal{E}  + n^2)$ | $O(n^2)$               |
| EdgedropExplainer | $O( \mathcal{E} )$       | $O( \mathcal{E}  + n)$ |
| ProxyExplainer    | $O( \mathcal{E}  + n^2)$ | $O(n^2)$               |

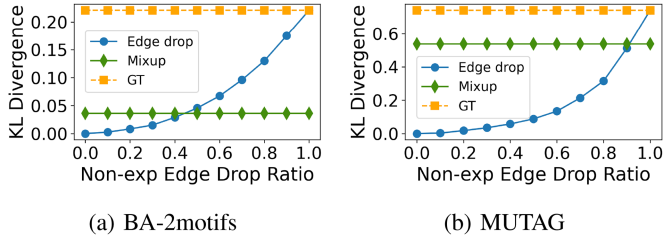


Fig. 3. Using BA-2motifs and MUTAG to demonstrate the KL divergence between the original graph and graphs generated by preserving  $G_{exp}$  and modifying  $G - G_{exp}$ . “Edge drop” represents the KL divergence between the original graph and the one obtained by preserving  $G_{exp}$  while removing edges from  $G - G_{exp}$  at varying ratios. “Mixup” replaces  $G - G_{exp}$  with the base subgraph sampled from other graphs. “GT” denotes the KL divergence between the original graph and the ground-truth explanation graph.

based on variational inference that learns to generate proxy graphs by explicitly optimizing both distribution alignment and explanation fidelity. We delineate our main contributions as follows:

- We systematically analyze and address the challenge of the OOD problem in explainable GNNs, which is pivotal to enhance the reliability and explainability of GNNs in real-world applications.
- We provide a theoretical analysis of the OOD problem inherent in the existing methods, and identify potential sources of distribution shifts between explanation subgraphs and original graphs. We also analyze how our proposed proxy graph approaches can help bridge this distributional gap while maintaining explanation fidelity, laying the groundwork for more reliable GNN explanations.
- We introduce both non-parametric and parametric methods to produce proxy graphs that not only reside within the distribution of the original data but also preserve the essential explanation information. This facilitates more precise and interpretable explanations in GNN applications.
- We adopt novel evaluation metrics that are resilient to distribution shifts and enhance their applicability in real-world contexts. Through comprehensive experiments on various real-world datasets, we substantiate the effectiveness of our proposed approach and demonstrate its utility and superiority in producing explanations.

## II. RELATED WORK

### A. Explainable Artificial Intelligence

The field of explainability in deep learning addresses the opacity of complex neural systems, especially in high-stakes applications. Explanation methods can be broadly classified into ante-hoc methods, which build inherently interpretable models, and post-hoc methods, which analyze pre-trained models to generate

explanations [1]. In the visual domain, ante-hoc methods such as CBM [24] and ProtoPNet [6] enhance transparency through concept and prototype layers, while post-hoc methods include feature activation mapping [27], Bayesian concept-based explanations [42], and hierarchical saliency clustering [4]. A recent line of research explores minimal visual explanations using submodular subset selection and search-based strategies [8], [9], [10], which find minimal image regions that preserve model predictions. While related to the minimality principle in GNN explanations, these methods rely on spatial partitions and discrete search that do not directly transfer to graph structures.

### B. GNN Explanations

The goal of explainability in GNNs is to ensure transparency in graph-based tasks. Gradient-based explanation methods such as GradCAM [33] and Gradient [33] perform well in continuous pixel spaces. However, they are less reliable for graphs, where discrete structures, permutation sensitivity, and off-manifold perturbations can lead to unstable gradients. This has driven the development of mask-based and optimization-based explainers in GNNs. Recent works have been directed toward elucidating the rationale behind GNN predictions. These explanation methods can be broadly classified into two categories: instance-level and model-level approaches [48]. In this study, we focus on instance-level explanations, which aim to clarify the specific reasoning behind individual predictions made by GNNs. For example, GNNExplainer [47] excludes certain edges and node features to observe the changes in prediction. However, its single-instance focus limits its applicability in providing a global understanding of the to-be-explained model [11]. PGExplainer [28], [29] introduces a parametric neural network to learn edge weights. Thus, once training is complete, it can explain new graphs without retraining. ReFine [43] integrates a pre-training phase that focuses on class comparisons and is fine-tuned to refine context-specific explanations. GStarX [52] assigns an importance score to each node by calculating the Hamiache and Navarro values of the structure to obtain explanation subgraphs. GFlowExplainer [25] uses a generator to construct a TD-like flow matching condition to learn a policy for generating explanations by adding nodes sequentially.

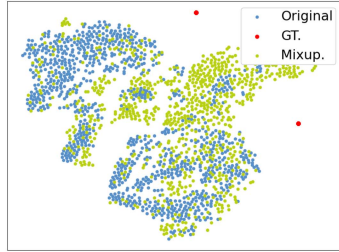
### C. Distribution Shifting in Explanations

The distribution shift problem in post-hoc explanations has been increasingly recognized in explainable AI [5], [35]. For example, FIDO [5] enhances image classifier explanations by focusing on relevant contextual details that align with the training data distribution. A recent study addresses distribution shift in image explanations by introducing a module that assesses the similarity between altered data and the original data distribution [35]. In the graph domain, an ad-hoc strategy to mitigate distribution shift is to initially reduce the size-constraint coefficient during the explanation process [16]. MixupExplainer [51] and RegExplainer [50] propose non-parametric solutions by mixing the explanation subgraph with a non-explanation part sampled from another graph.

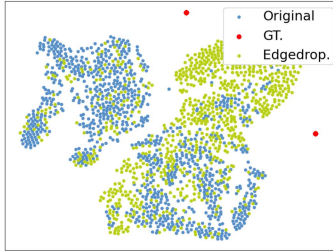
However, these methods construct the non-explanation component independently of the given explanation subgraph. In

TABLE III  
EXPLANATION ACCURACY (AUC-ROC) FOR GRAPH CLASSIFICATION WITH GROUND-TRUTH ANNOTATIONS

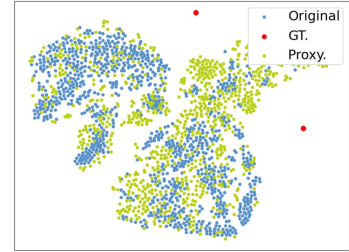
|                   | MUTAG                    | Benzene                  | Alkane-Carbonyl          | Fluoride-Carbonyl        | BA-2motifs               | BA-3motifs               |
|-------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|
| GradCAM           | 0.727 $\pm$ 0.000        | 0.740 $\pm$ 0.000        | 0.448 $\pm$ 0.000        | 0.694 $\pm$ 0.000        | 0.714 $\pm$ 0.000        | 0.709 $\pm$ 0.000        |
| GNExplainer       | 0.682 $\pm$ 0.009        | 0.485 $\pm$ 0.001        | 0.551 $\pm$ 0.003        | 0.574 $\pm$ 0.002        | 0.644 $\pm$ 0.007        | 0.511 $\pm$ 0.002        |
| PGExplainer       | 0.832 $\pm$ 0.032        | 0.793 $\pm$ 0.054        | 0.660 $\pm$ 0.036        | 0.702 $\pm$ 0.018        | 0.734 $\pm$ 0.117        | 0.796 $\pm$ 0.010        |
| ReFine            | 0.612 $\pm$ 0.004        | 0.606 $\pm$ 0.002        | 0.768 $\pm$ 0.001        | 0.571 $\pm$ 0.000        | 0.698 $\pm$ 0.001        | 0.629 $\pm$ 0.005        |
| MixupExplainer    | 0.863 $\pm$ 0.103        | 0.611 $\pm$ 0.032        | 0.811 $\pm$ 0.006        | 0.706 $\pm$ 0.013        | 0.906 $\pm$ 0.059        | 0.859 $\pm$ 0.019        |
| EdgedropExplainer | 0.864 $\pm$ 0.107        | 0.842 $\pm$ 0.048        | <b>0.946</b> $\pm$ 0.000 | 0.609 $\pm$ 0.028        | 0.672 $\pm$ 0.139        | 0.623 $\pm$ 0.060        |
| ProxyExplainer    | <b>0.977</b> $\pm$ 0.009 | <b>0.845</b> $\pm$ 0.036 | 0.934 $\pm$ 0.005        | <b>0.758</b> $\pm$ 0.068 | <b>0.935</b> $\pm$ 0.008 | <b>0.960</b> $\pm$ 0.008 |



(a) MixupExplainer



(b) EdgedropExplainer



(c) ProxyExplainer

Fig. 4. Visualizations of the distribution shifts issue with t-SNE on BA-2motifs dataset.

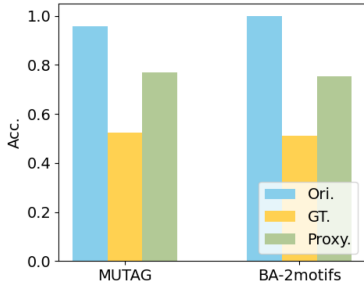


Fig. 5. Classification accuracy on original graphs, ground-truth explanation graphs, and proxy graphs.

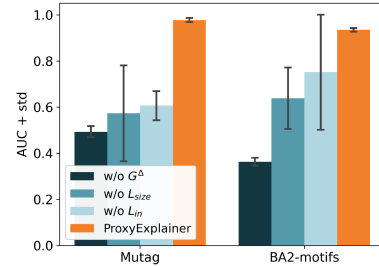


Fig. 6. Ablation study on MUTAG and BA-2motifs datasets.

real-world graphs, the remaining structure is often correlated with the explanation due to domain-specific constraints such as chemical valence in molecules [21], [36], [39] or triangle closure in social networks [46]. Unconditional cross-graph sampling can therefore produce mixed graphs that deviate from the training distribution despite being designed to reduce distribution shift. Our work departs from this paradigm by providing a formal analysis of explanation-induced structural distribution shift and introducing proxy graphs whose construction is explicitly conditioned on the explanation subgraph, ensuring joint consistency with both the explanation and the training distribution.

### III. NOTATIONS AND PROBLEM FORMULATION

We denote a graph  $G$  by a triplet  $(\mathcal{V}, \mathcal{E}, \mathbf{X})$ , where  $\mathcal{V} = \{v_1, v_2, \dots, v_n\}$  is the node set,  $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$  is the edge set, and  $\mathbf{X} \in \mathbb{R}^{n \times d}$  is the node feature matrix, where  $d$  is the feature dimension. The  $i$ -th row of  $\mathbf{X}$  is the feature vector associated with node  $v_i$ . The adjacency matrix of  $G$  is denoted by  $\mathbf{A} \in \{0, 1\}^{n \times n}$ , which is determined by the edge set  $\mathcal{E}$

such that  $A_{ij} = 1$  if  $(v_i, v_j) \in \mathcal{E}$ , and  $A_{ij} = 0$ , otherwise. We focus on the graph classification task, as the node classification problem can be converted to a computation graph classification problem [28], [47]. In the graph classification task, each graph  $G$  is associated with a label  $Y \in \mathcal{Y}$ . We denote the underlying probability distribution by  $P_{G,Y}$ , and use  $\mathcal{G}$  to denote the support of  $P_G$ . A GNN classifier is a mapping  $f: \mathcal{G} \rightarrow \Delta_{\mathcal{Y}}$ , where  $\Delta_{\mathcal{Y}}$  denotes the probability simplex on  $\mathcal{Y}$ . Following the existing works [19], [28], [47], [48], the explanation methods under consideration in this paper are model/task agnostic which treat GNN models as opaque systems, *i.e.*, the so-called *post-hoc*, *instance-level* explanation methods. The following statement provides an informal description of the post-hoc instance-level explanation problem. The notion is made more precise in the subsequent sections.

**Problem 1. (Post-hoc Instance-level GNN Explanation):** Given a to-be-explained GNN model  $f(\cdot)$ , the objective in post-hoc instance-level explanation is to obtain an explanation function(explainer)  $\Psi(\cdot)$  which, given a graph  $G \in \mathcal{G}$ , finds a subgraph  $G_{exp} \subseteq G$  that is “minimal” and “sufficient”.

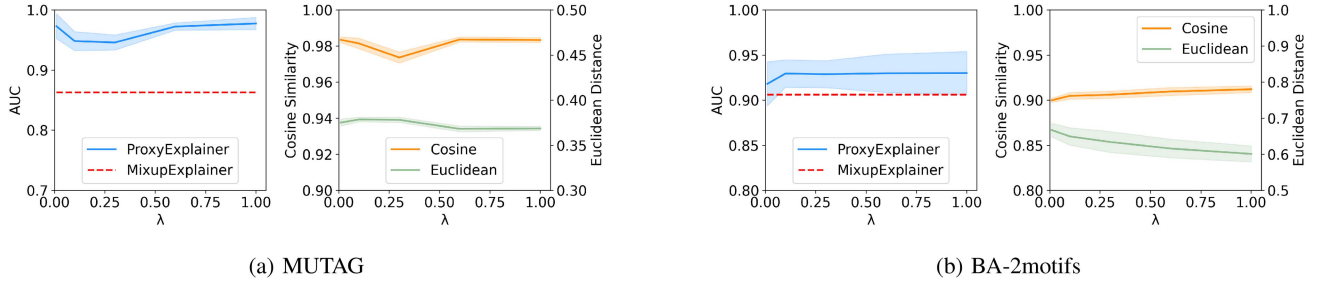


Fig. 7. Parameter analysis of  $\lambda$  on two datasets. The left side of each graph shows the explanation performance. The right side shows the Distance Analysis between  $\mathbf{h}$  and  $\hat{\mathbf{h}}$ .

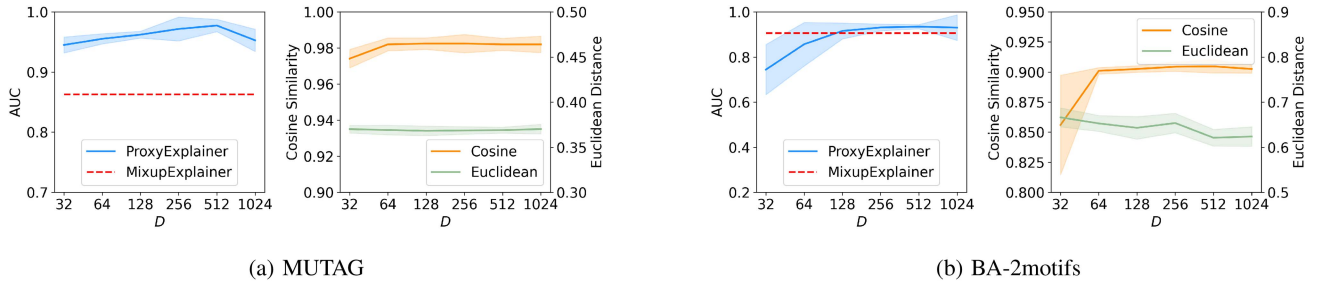


Fig. 8. Parameter analysis of the dimension of node latent embeddings. The left side of each graph shows the explanation performance and the right side displays the Cosine score and Euclidean distance between  $\mathbf{h}$  and  $\hat{\mathbf{h}}$ .

#### IV. INFORMATION-THEORETIC FRAMEWORK AND OOD PROBLEM

##### A. Sufficient and Minimal Explanations

In the context of GNNs, explainability refers to the ability to identify the most influential subgraph  $G_{exp}$  in the input graph  $G$  that determines the model's prediction  $f(G)$ . A *sufficient* subgraph  $G_{exp}$  is the one that contains most of the pertinent statistics of the input graph  $G$  to output label  $\hat{Y}$  produced by  $f(G)$ . The posterior distribution of the output label  $\hat{Y}$  should not change significantly if we are given the explanation subgraph  $G_{exp}$  instead of the complete graph, i.e.  $d_{TV}(P_{\hat{Y}|G}, P_{\hat{Y}|G_{exp}}) \approx 0$ , where  $d_{TV}$  denotes the total variation distance. Additionally, an explanation is called *minimal*, if its number of edges is minimized subject to the aforementioned sufficiency constraints. Formally, the sufficiency and minimality of explanations are formulated as below.

**Definition 1. (Model Explainability):** Given a sufficiency parameter  $\zeta \in [0, 1]$  and a minimality parameter  $s \in \mathbb{N}$ , a model  $f: \mathcal{G} \rightarrow \Delta_{\mathcal{Y}}$  for the task  $P_{G,Y}$  is called  $(s, \zeta)$ -explainable if there exists an explanation function  $\Psi(\cdot)$  such that:<sup>1</sup>

$$i) \mathbb{E}_G(d_{TV}(P_{\hat{Y}|G}, P_{\hat{Y}|G_{exp} \subseteq G})) \leq \zeta, \quad (1)$$

$$ii) \mathbb{E}_G(|\mathcal{E}_{exp}|) \leq s, \quad (2)$$

where  $G_{exp} = \Psi(G) = (\mathcal{V}_{exp}, \mathcal{E}_{exp})$  represents the explanation subgraph,  $\hat{Y}$  is generated according to the probability distribution  $f(G)$ , and  $f(\cdot)$  is the to-be-explained model.

<sup>1</sup>One could define task explainability — instead of model explainability — by replacing the model output  $\hat{Y}$  with the ground-truth label  $Y$  in (1) and (2). The relation between model and task explainability has been studied in [53].

A standard objective function is to fix the minimality parameter  $s$ , and search for the *optimal* explainer  $\Psi^*(\cdot)$  achieving the smallest sufficiency parameter  $\zeta$ . That is,

$$\Psi^* = \arg \min_{\Psi: \mathcal{G} \rightarrow \mathcal{G}} \mathbb{E}_G(d_{TV}(P_{\hat{Y}|G}, P_{\hat{Y}|G_{exp} \subseteq G})),$$

$$\text{s.t. } \mathbb{E}(|\mathcal{E}_{exp}|) \leq s.$$

We note that by definition  $P_{\hat{Y}|G} = f(G)$ . So, the optimization can be written in the following form:

$$\Psi^* = \arg \min_{\Psi: \mathcal{G} \rightarrow \mathcal{G}} \mathbb{E}_G(d_{TV}(f(G), P_{\hat{Y}|G_{exp} \subseteq G})),$$

$$\text{s.t. } \mathbb{E}(|\mathcal{E}_{exp}|) \leq s. \quad (3)$$

The challenge to finding  $\Psi^*(\cdot)$  is to find computationally efficient methods to estimate  $P_{\hat{Y}|G_{exp} \subseteq G}$ . Existing loss functions and evaluation metrics for training explainers can be viewed as different methods that provide an estimate of the above quantities as described in the following.

##### B. Connection to Graph Information Bottleneck

Optimizing the graph information bottleneck (GIB) loss function — commonly used to train explainers such as GNNExplainer [47] and PGExplainer [28] — can be viewed as estimating a tangent line to the achievable  $(s, \zeta)$  parameter space for a given model, where the slope of the tangent line is determined by the GIB parameter  $\alpha \in [0, 1]$ . To elaborate, the GIB loss function is defined as follows:<sup>2</sup>

$$\min_{\Psi: \mathcal{G} \rightarrow \mathcal{G}} I(G; \Psi(G)) - \alpha I(\hat{Y}; \Psi(G)). \quad (4)$$

<sup>2</sup>Following the standard notational conventions in the literature, we have defined  $I(G; \Psi(G)) = H(G) - H(G|\Psi(G) \subseteq G)$  and  $I(Y; \Psi(G)) = H(Y) - H(Y|\Psi(G) \subseteq G)$ .

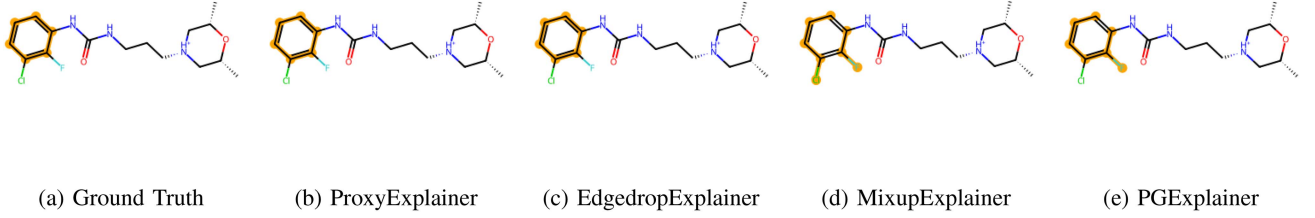


Fig. 9. Visualization of the explanation results from different explanation models on Benzene dataset. The generated explanations are highlighted with bold orange edges.

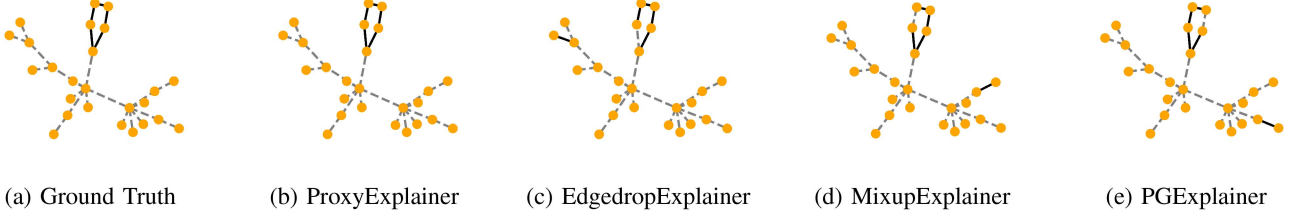


Fig. 10. Visualization of the explanation results from different explanation models on BA-2motifs dataset. The generated explanations are highlighted with bold black edges.

The first term is the mutual information between the original graph  $G$  and the explanation subgraph  $\Psi(G)$  and the second term is the negative mutual information of  $Y$  and the explanation subgraph  $\Psi(G)$ . At a high level, the first term encourages detecting a small and dense subgraph for explanation, and the second term requires that the explanation subgraph is label-preserving. We will show that the first term  $I(G; \Psi(G))$  provides an estimate of the minimality parameter in Definition 1 and the second term  $I(\hat{Y}; \Psi(G))$  provides an estimate of the sufficiency parameter.

Let us first focus on the  $I(G; \Psi(G))$  term. To keep the analysis tractable, we adopt the edge-independence assumption commonly used in prior works [28], [32]. Specifically, we assume that each edge in the original graph  $G$  exists independently. For the prior distribution over explanation subgraphs, we further assume a uniform edge probability  $r \in [0, 1]$ , such that  $P(G) = r^{|\mathcal{E}_G|} (1-r)^{\binom{|V|}{2} - |\mathcal{E}_G|}$ , where  $\binom{|V|}{2} = \frac{|V|(|V|-1)}{2}$  is the number of possible edges.

Under this assumption, the mutual information  $I(G; \Psi(G))$  can be decomposed as  $I(G; \Psi(G)) = H(G) - H(G|\Psi(G) \subseteq G)$ . The first term  $H(G)$  represents the entropy of the original graph. Since each of the  $\binom{|V|}{2}$  possible edges independently follows a Bernoulli( $r$ ) distribution, we have:  $H(G) = \binom{|V|}{2} h_b(r)$ , where  $h_b(r) = -r \log r - (1-r) \log(1-r)$  is the binary entropy function. The second term  $H(G|\Psi(G) \subseteq G)$  is the conditional entropy of  $G$  given that the explanation subgraph  $\Psi(G)$  is contained in  $G$ . Once the edges in  $\Psi(G)$  are fixed, the remaining  $\binom{|V|}{2} - |\mathcal{E}_{\Psi(G)}|$  edges still follow independent Bernoulli( $r$ ) distributions. Thus:  $H(G|\Psi(G) \subseteq G) = (\binom{|V|}{2} - |\mathcal{E}_{\Psi(G)}|) h_b(r)$ . Combining these results:

$$I(G; \Psi(G)) = \mathbb{E}_G[|\mathcal{E}_{\Psi(G)}|] \cdot h_b(r).$$

This shows that minimizing  $I(G; \Psi(G))$  is equivalent to minimizing the expected number of edges in the explanation subgraph, scaled by a constant factor  $h_b(r)$ . This provides a formal justification for the minimality criterion.

Next, let us focus on the  $I(\hat{Y}; \Psi(G))$  term. We note that:

$$\begin{aligned} I(\hat{Y}; \Psi(G)) &= H(\hat{Y}) + \mathbb{E}_{\hat{Y}, G}(\log P_{\hat{Y}|\Psi(G) \subseteq G}) \\ &= H(\hat{Y}) - H(\hat{Y}|G) - \mathbb{E}_G(KL(P_{\hat{Y}|G}, P_{\hat{Y}|\Psi(G) \subseteq G})) \\ &\stackrel{(a)}{\leq} H(\hat{Y}) - H(\hat{Y}|G) - 2\mathbb{E}_G(d_{TV}^2(P_{\hat{Y}|G}, P_{\hat{Y}|\Psi(G) \subseteq G})) \\ &\stackrel{(b)}{\leq} H(\hat{Y}) - H(\hat{Y}|G) - 2\mathbb{E}_G^2(d_{TV}(P_{\hat{Y}|G}, P_{\hat{Y}|\Psi(G) \subseteq G})), \end{aligned}$$

where  $KL(\cdot, \cdot)$  denotes the Kullback-Liebler (KL) divergence, in (a), we have used Pinsker's type inequality (e.g., [12]), and in (b) we have used Jensen's inequality. Furthermore, the inequality becomes tighter as  $P_{\hat{Y}|\Psi(G) \subseteq G}$  gets closer to  $P_{\hat{Y}|G}$ , which means the explainer is accurate.

The above analysis reveals that minimizing the GIB objective in (4) is approximately equivalent to:

$$\min_{\Psi: G \rightarrow G} \mathbb{E}_G(|\mathcal{E}_{exp}|) \cdot h_b(r) + 2\alpha \mathbb{E}_G^2(d_{TV}(P_{\hat{Y}|G}, P_{\hat{Y}|\Psi(G) \subseteq G})).$$

This establishes a connection between the GIB loss and our sufficiency-minimality framework in Definition 1: the first term enforces minimality (small explanation size), while the second term enforces sufficiency (prediction preservation). For any given  $\alpha \in [0, 1]$ , this optimization yields a point on the boundary of achievable  $(s, \zeta)$  parameters.

### C. The Structural Distribution Shift Problem

As discussed in the prequel, a key step in quantifying the quality of explanations — which in turn is crucial in the design of loss functions and evaluation metrics — is to estimate the conditional probability  $P(\hat{Y}|\Psi(G) \subseteq G)$  of the output label given that  $\Psi(G)$  is in  $G$ . In prior works, the estimation of  $P(\hat{Y}|\Psi(G) \subseteq G)$  is typically achieved by applying the to-be-explained model  $f(\cdot)$  to the input graph  $\Psi(G)$  [32], [47]. A critical assumption in these approaches involves the GNN model's ability to accurately

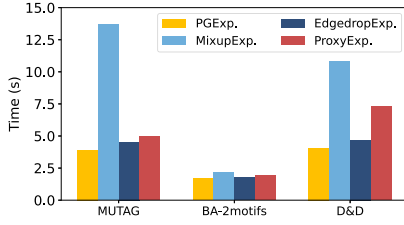


Fig. 11. Per-epoch training time of explanation methods.

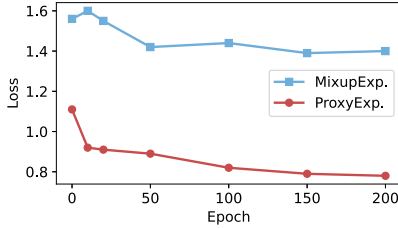


Fig. 12. Training loss curves of the explainers over training epochs on BA-2motifs.

predict candidate explanation subgraphs  $G_{exp}$ . This assumption, however, overlooks the OOD problem, where the distribution of explanation subgraphs significantly deviates from that of the original training graphs [2], [15], [16], [17], [51], [53].

## V. ROBUST OBJECTIVE WITH PROXY GRAPHS

In this section, we introduce the concept of *proxy* graphs as an alternative solution to estimating  $P(\hat{Y}|\Psi(G) \subseteq G)$ . Specifically, we wish to generate a proxy graph  $\tilde{G}$  drawn from the distribution  $P_{G|\Psi(G) \subseteq G}$  and reformulate the estimation of  $P(\hat{Y}|\Psi(G) \subseteq G)$  by marginalizing over the distribution of proxy graphs as follows:

$$P(\hat{Y}|\Psi(G) \subseteq G) = \mathbb{E}_{\tilde{G}|\Psi(G) \subseteq G} [P(\hat{Y}|\tilde{G})]. \quad (5)$$

To facilitate the maximization of the likelihood as outlined in (5), we further approximate  $P(\tilde{G}|\Psi(G) \subseteq G)$  with a parameterized function, denoted as  $Q_\phi(\tilde{G}|\Psi(G) \subseteq G)$ . The formal representation is thus given by

$$P(\hat{Y}|\Psi(G) \subseteq G) = \mathbb{E}_{Q_\phi} [P(\hat{Y}|\tilde{G})], \quad (6)$$

where  $\phi$  denotes model parameters and  $\mathbb{E}_{Q_\phi}$  represents the expectation with respect to  $Q_\phi(\tilde{G}|\Psi(G) \subseteq G)$ .

With this proxy graph formulation, we can construct a robust objective for training the explainer. Recall from (3) that the optimal explainer minimizes the divergence between model predictions on the original graph and on the explanation subgraph, subject to a size constraint. Substituting our proxy-based estimator from (6), the robust objective becomes:

$$\begin{aligned} \min_{\Psi: \tilde{G} \rightarrow \mathcal{G}} \quad & \mathbb{E}_G \left( d_{TV} \left( f(G), \mathbb{E}_{Q_\phi} [f(\tilde{G})] \right) \right) + \alpha \mathbb{E}_G (|\mathcal{E}_{exp}|), \\ \text{s.t.} \quad & Q_\phi(\tilde{G}|\Psi(G) \subseteq G) \approx P(\tilde{G}|\Psi(G) \subseteq G), \end{aligned} \quad (7)$$

where the hyper-parameter  $\alpha > 0$  controls the trade-off between sufficiency and minimality.

### A. Bi-Level Optimization.

In (7), we formulate a joint optimization loss function that aims to identify the optimal explanation alongside its corresponding proxy graphs. Building upon this, we refine the framework by replacing the first constraint with a distributional distance measure, specifically the KL divergence, between  $Q_\phi(\tilde{G}|\Psi(G) \subseteq G)$  and  $P_{\tilde{G}}$ . This leads to the development of a bi-level optimization model. The model is expressed as follows:

$$\begin{aligned} \Psi^* = \arg \min_{\Psi: \mathcal{G} \rightarrow \mathcal{G}} \quad & \mathbb{E}_G \left( d_{TV} \left( f(G), \mathbb{E}_{Q_{\phi^*}} [f(\tilde{G})] \right) \right) \\ & + \alpha \mathbb{E}_G (|\mathcal{E}_{exp}|), \\ \text{where} \quad & \\ \phi^* = \arg \min_{\phi} \quad & \text{KL}(Q_\phi(\tilde{G}|\Psi(G) \subseteq G) \| P(\tilde{G}|\Psi(G) \subseteq G)). \end{aligned} \quad (8)$$

*Derivation of Outer Optimization:* The outer optimization involves two terms: the sufficiency term  $d_{TV}(f(G), \mathbb{E}_{Q_\phi}[f(\tilde{G})])$  and the minimality term  $\mathbb{E}_G(|\mathcal{E}_{exp}|)$ . We derive tractable surrogates for both.

For the sufficiency term, we note that the total variation distance is upper bounded by KL divergence via Pinsker's inequality, i.e.,  $d_{TV}(P, Q) \leq \sqrt{\frac{1}{2} KL(P\|Q)}$ . Thus, minimizing  $KL(f(G) \| \mathbb{E}_{Q_\phi}[f(\tilde{G})])$  provides an upper bound on  $d_{TV}$ . In practice, we use the cross-entropy loss

$$\mathcal{L}_{\text{suff}} = -\mathbb{E}_{\tilde{G} \sim Q_\phi} [\log f(\tilde{G})_{\hat{y}}],$$

where  $\hat{y}$  is the predicted class for the original graph  $G$ .

For the minimality term, the theoretical objective is to minimize  $\mathbb{E}_G(|\mathcal{E}_{exp}|)$ . Under the edge-independence assumption with selection probabilities  $\pi'_{uv}$ , this equals  $\sum_{(u,v) \in \mathcal{E}} \pi'_{uv}$ . In practice, we use Gumbel-Softmax relaxation for differentiable training, which requires an additional regularizer to encourage discrete edge selections. Following [28], we adopt the KL divergence against a sparse Bernoulli prior

$$\mathcal{L}_{\text{size}} = \sum_{(u,v) \in \mathcal{E}} \pi'_{uv} \log \frac{\pi'_{uv}}{r} + (1 - \pi'_{uv}) \log \frac{1 - \pi'_{uv}}{1 - r}, \quad (9)$$

where  $r$  controls the sparsity level. The final loss for the outer optimization combines both terms as follows:

$$\mathcal{L}_{\text{out}} = \mathcal{L}_{\text{suff}} + \alpha \mathcal{L}_{\text{size}}. \quad (10)$$

*Derivation of Inner Optimization:* The inner optimization minimizes the distributional distance between  $Q_\phi(\tilde{G}|\Psi(G))$  and  $P(\tilde{G}|\Psi(G) \subseteq G)$ . Under the edge-independence assumption, this reduces to a weighted cross-entropy loss between  $\tilde{G}$  and  $G$ :

$$\mathcal{L}_{\text{in}} = \frac{\beta}{|\mathcal{E}|} \sum_{(u,v) \in \mathcal{E}} \log(\tilde{p}_{uv}) + \frac{1}{|\mathcal{E}|} \sum_{(u,v) \in \bar{\mathcal{E}}} \log(1 - \tilde{p}_{uv}), \quad (11)$$

where  $\tilde{p}_{uv}$  is the predicted probability of edge  $(u, v)$  in the proxy graph  $\tilde{G}$ , and  $\beta$  balances connected and unconnected node pairs.

TABLE IV  
EXPLANATION FIDELITY

|              | MUTAG                       |                               | Benzene                     |                               | Alkane-Carbonyl             |                               |
|--------------|-----------------------------|-------------------------------|-----------------------------|-------------------------------|-----------------------------|-------------------------------|
|              | $Fid_{\alpha_1,+} \uparrow$ | $Fid_{\alpha_2,-} \downarrow$ | $Fid_{\alpha_1,+} \uparrow$ | $Fid_{\alpha_2,-} \downarrow$ | $Fid_{\alpha_1,+} \uparrow$ | $Fid_{\alpha_2,-} \downarrow$ |
| GradCAM      | 0.004 $\pm$ 0.000           | 0.162 $\pm$ 0.000             | 0.103 $\pm$ 0.000           | 0.066 $\pm$ 0.000             | 0.017 $\pm$ 0.000           | 0.165 $\pm$ 0.000             |
| GNNExp.      | 0.031 $\pm$ 0.001           | 0.148 $\pm$ 0.001             | 0.060 $\pm$ 0.001           | 0.113 $\pm$ 0.001             | 0.011 $\pm$ 0.001           | 0.145 $\pm$ 0.002             |
| PGExp.       | 0.034 $\pm$ 0.011           | 0.148 $\pm$ 0.005             | <u>0.128</u> $\pm$ 0.017    | 0.051 $\pm$ 0.012             | 0.019 $\pm$ 0.003           | 0.137 $\pm$ 0.004             |
| ReFine       | 0.003 $\pm$ 0.000           | 0.160 $\pm$ 0.001             | 0.077 $\pm$ 0.001           | 0.088 $\pm$ 0.001             | 0.024 $\pm$ 0.001           | 0.159 $\pm$ 0.001             |
| MixupExp.    | <u>0.037</u> $\pm$ 0.006    | <u>0.146</u> $\pm$ 0.003      | 0.085 $\pm$ 0.024           | 0.078 $\pm$ 0.013             | 0.026 $\pm$ 0.002           | 0.138 $\pm$ 0.001             |
| EdgedropExp. | 0.034 $\pm$ 0.010           | <u>0.146</u> $\pm$ 0.004      | <b>0.131</b> $\pm$ 0.008    | <u>0.047</u> $\pm$ 0.003      | <b>0.031</b> $\pm$ 0.001    | <u>0.134</u> $\pm$ 0.001      |
| ProxyExp.    | <b>0.040</b> $\pm$ 0.002    | <b>0.145</b> $\pm$ 0.001      | <b>0.131</b> $\pm$ 0.013    | <b>0.046</b> $\pm$ 0.008      | <u>0.028</u> $\pm$ 0.003    | <b>0.132</b> $\pm$ 0.002      |
|              | Fluoride-Carbonyl           |                               | BA-2motifs                  |                               | BA-3motifs                  |                               |
|              | $Fid_{\alpha_1,+} \uparrow$ | $Fid_{\alpha_2,-} \downarrow$ | $Fid_{\alpha_1,+} \uparrow$ | $Fid_{\alpha_2,-} \downarrow$ | $Fid_{\alpha_1,+} \uparrow$ | $Fid_{\alpha_2,-} \downarrow$ |
| GradCAM      | 0.110 $\pm$ 0.000           | <b>0.028</b> $\pm$ 0.000      | 0.072 $\pm$ 0.000           | <u>0.107</u> $\pm$ 0.000      | 0.074 $\pm$ 0.000           | 0.032 $\pm$ 0.000             |
| GNNExp.      | 0.046 $\pm$ 0.001           | 0.132 $\pm$ 0.001             | 0.057 $\pm$ 0.002           | 0.132 $\pm$ 0.001             | 0.077 $\pm$ 0.001           | 0.029 $\pm$ 0.001             |
| PGExp.       | 0.108 $\pm$ 0.007           | 0.081 $\pm$ 0.009             | 0.065 $\pm$ 0.017           | 0.126 $\pm$ 0.009             | 0.069 $\pm$ 0.008           | 0.027 $\pm$ 0.007             |
| ReFine       | 0.058 $\pm$ 0.001           | 0.076 $\pm$ 0.001             | 0.060 $\pm$ 0.005           | 0.125 $\pm$ 0.001             | <b>0.087</b> $\pm$ 0.001    | <u>0.025</u> $\pm$ 0.001      |
| MixupExp.    | <u>0.109</u> $\pm$ 0.004    | 0.074 $\pm$ 0.004             | 0.073 $\pm$ 0.005           | 0.112 $\pm$ 0.003             | 0.080 $\pm$ 0.003           | 0.039 $\pm$ 0.002             |
| EdgedropExp. | 0.053 $\pm$ 0.002           | 0.087 $\pm$ 0.002             | <u>0.077</u> $\pm$ 0.020    | 0.116 $\pm$ 0.007             | 0.079 $\pm$ 0.005           | 0.030 $\pm$ 0.005             |
| ProxyExp.    | <b>0.112</b> $\pm$ 0.021    | <u>0.049</u> $\pm$ 0.028      | <b>0.086</b> $\pm$ 0.003    | <b>0.106</b> $\pm$ 0.002      | <u>0.085</u> $\pm$ 0.004    | <b>0.022</b> $\pm$ 0.003      |

## VI. THE PROXYEXPLAINER

In this section, we propose two complementary implementations of the proposed ProxyExplainer framework, each designed to support different application scenarios. The first implementation utilizes explanation-preserving edge dropping, offering a non-parametric solution with theoretical guarantees for distribution preservation. The second implementation employs variational autoencoders, providing a more flexible parametric approach capable of handling complex graph structures and diverse data distributions. These implementations enable ProxyExplainer to address the OOD problem while accommodating various real-world graph analysis requirements. In the following subsections, we first present the edge dropping-based approach, followed by the detailed architecture of our variational autoencoder-based solution.

## A. The Explainer

For the sake of optimization, we follow the existing works to relax the elements in  $\mathbf{A}$  from binary values to continuous values in the range  $[0, 1]$  [28], [29]. We adopt a generative explainer due to its effectiveness and efficiency [11]. Specifically, we decompose the to-be-explained model  $f(\cdot)$  into two functions: 1)  $f_{\text{enc}}(\cdot)$ , which learns node representations, and 2)  $f_{\text{cls}}(\cdot)$ , which predicts graph labels based on node embeddings. Formally, we have  $\mathbf{Z} = f_{\text{enc}}(\mathbf{X}, \mathbf{A})$  and  $\hat{Y} = f_{\text{cls}}(\mathbf{Z})$ , where  $\mathbf{Z}$  is the node representation matrix and  $\hat{Y}$  is the prediction. Typically,  $f_{\text{cls}}(\cdot)$  consists of a pooling layer followed by a classification layer and  $f_{\text{enc}}(\cdot)$  consists of the other layers. Following the independence assumption, we approximate Bernoulli distributions with binary concrete distributions [20], [31]. Specifically, the probability of sampling an edge  $(u, v)$  is computed by an MLP parameterized by  $\psi$ , denoted as  $g_{\psi}(\cdot)$ . Formally, we have

$$\begin{aligned}
 \omega_{uv} &= g_{\psi}([\mathbf{z}_u; \mathbf{z}_v]), \\
 \epsilon &\sim \text{Uniform}(0, 1), \\
 A'_{uv} &= \sigma((\log \epsilon - \log(1 - \epsilon) + \omega_{uv})/\tau), \quad (12)
 \end{aligned}$$

where  $[\mathbf{z}_u; \mathbf{z}_v]$  is the concatenation of node representations  $\mathbf{z}_u$  and  $\mathbf{z}_v$ ,  $\epsilon$  is an independent variable,  $\sigma(\cdot)$  is the Sigmoid function, and  $\tau$  is a temperature hyper-parameter for approximation. According to [28], the parameter  $\pi'_{uv}$  of Bernoulli distribution in (9) can be obtained by  $\pi'_{uv} = \frac{\exp(\omega_{uv})}{1 + \exp(\omega_{uv})}$ .

## B. Generating Proxy Via Edge Dropping

Following our notation from the previous section, we first implement our proxy graph framework through a straightforward yet effective edge-dropping mechanism. This implementation, denoted as EdgedropExplainer, stems from a fundamental observation: while the explanation subgraph  $G_{\text{exp}}$  must be preserved to maintain fidelity, the non-explanation components can be modified to better align the proxy graph with the original distribution  $P_G$ . To validate this insight, we conduct experiments on the BA-2motifs and MUTAG datasets, as illustrated in Fig. 3. The ground-truth explanation graph (denoted as GT in the figure) exhibits a significant KL divergence from the original graph distribution, as shown by the yellow dashed line. Fig. 3 also includes a Mixup method that replaces  $G - G_{\text{exp}}$  with a residual subgraph sampled from other graphs. On the synthetic dataset BA-2motifs, Mixup shows a small KL divergence, suggesting that cross-graph replacement remains approximately distributionally consistent in this controlled setting. In contrast, on the real-world dataset MUTAG, Mixup induces a substantially larger KL divergence, indicating a pronounced distribution mismatch. This suggests that the implicit independence assumption underlying Mixup may hold in synthetic settings but breaks down in real-world graphs, where the residual structure is often conditionally dependent on  $G_{\text{exp}}$  through co-occurring structural patterns. As a result, unconditional cross-graph splicing can produce OOD samples.

To address this distributional gap, we introduce an explanation-preserving augmentation strategy that maintains  $G_{\text{exp}}$  while systematically removing edges from  $G - G_{\text{exp}}$  at varying ratios. Our experiments demonstrate that the KL

**Algorithm 1:** Algorithm of EdgedropExplainer.

---

```

1: Input: A set of graphs  $\mathcal{G} = \{G_i\}_{i=0}^N$ , with each
    $G_i = (\mathcal{V}_i, \mathcal{E}_i, \mathbf{X}_i)$ , a pretrain to-be-explained model
    $f(\cdot)$ , epochs  $E$ , dropping ratio  $p$ .
2: Initialize an explainer function  $\Psi(\cdot)$ 
3: epoch  $\leftarrow 0$ 
4: while epoch  $< E$  do
5:   for  $G_i \in \mathcal{G}$  do
6:      $G_{i,exp} \leftarrow \Psi(G_i)$ 
7:      $G_i^\Delta \leftarrow G_i - G_{i,exp}$ 
8:      $\mathcal{E}_i^\Delta = \{e \mid e \in \Delta\mathcal{E}_i, \text{Bernoulli}(1-p) = 1\}$ 
9:     Compute explainer loss  $\mathcal{L}_{exp}$ 
10:    Update parameters in the explainer with
        backpropagation
11:   end for
12:   epoch  $\leftarrow$  epoch + 1
13: end while

```

---

divergence increases monotonically with the edge drop ratio, indicating that aggressive edge removal substantially alters the original graph structure and its distributional properties.

A key finding from these experiments is that moderate edge dropping from  $G - G_{exp}$  maintains a relatively small KL divergence while preserving the essential explanation information contained in  $G_{exp}$ . This observation motivates EdgedropExplainer to generate proxy graphs  $\tilde{G}$  that better align with  $P_G$  by combining  $G_{exp}$  with a carefully sampled subset of non-explanation edges.

As shown in Fig. 2(a), EdgedropExplainer applies the explainer to obtain  $G_{exp}$  from the input graph  $G$ , optimizing it according to (8). We then compute the complement subgraph  $G - G_{exp}$  and sample its edges with probability  $1 - p$  to construct  $G^\Delta = (\mathcal{V}, \mathcal{E}^\Delta, \mathbf{X})$ . We define the edge set  $\mathcal{E}^\Delta$  as  $\text{Sample}(\mathcal{E} - \mathcal{E}_{exp}, 1 - p)$ . Correspondingly, the adjacency matrix  $\mathbf{A}^\Delta$  is obtained by  $\text{Sample}(\mathbf{A} - \mathbf{A}_{exp}, 1 - p)$ . The final proxy graph  $\tilde{G}$  is formed by combining  $G_{exp}$  and  $G^\Delta$ , ensuring both explanation preservation and distributional alignment with  $P_G$ . This non-parametric construction effectively solves the inner optimization in (8) without learnable parameters, and only the outer optimization requires gradient computation through the explainer's Gumbel-Softmax relaxation. Algorithm 1 details this procedure.

### C. Generating Proxy With Variational AutoEncoder

In this section, we propose an alternative approach, named ProxyExplainer, which extends the edge-dropping mechanism by introducing a learnable structure generation process. Unlike EdgedropExplainer, which modifies  $G - G_{exp}$  by randomly removing edges, ProxyExplainer learns a structured representation of the non-explanation subgraph through Variational Graph Autoencoders (VGAE), as shown in Fig. 2(b). Following the same explainer framework as in EdgedropExplainer, we obtain  $G_{exp}$ , optimized through the outer objective in (8). The proxy graph generator then generates an in-distribution proxy graph, which is optimized with the inner objective.

**Algorithm 2:** Algorithm of ProxyExplainer.

---

```

1: Input: A set of graphs  $\mathcal{G} = \{G_i\}_{i=0}^N$ , with each
    $G_i = (\mathcal{V}_i, \mathcal{E}_i, \mathbf{X}_i)$ , a pretrain to-be-explained model
    $f(\cdot)$ , hyper parameters  $\alpha, \lambda, M$ , epochs  $E$ .
2: Initialize an explainer function  $\Psi(\cdot)$ 
3: epoch  $\leftarrow 0$ 
4: while epoch  $< E$  do
5:   for  $G_i \in \mathcal{G}$  do
6:      $G_{i,exp} \leftarrow \Psi(G_i)$ 
7:      $G_i^\Delta \leftarrow G_i - G_{i,exp}$ 
8:     Compute  $\tilde{\mathbf{A}}_{i,exp}$  with (13)
9:     Compute  $\mathbf{Z}^\Delta$  and  $\tilde{\mathbf{A}}_i^\Delta$  with (15)
10:    Compute proxy loss  $\mathcal{L}_{gen}$  with (17)
11:    Update parameters in proxy graph generator with
        backpropagation
12:    if epoch %  $M == 0$  then
13:      Compute explainer loss  $\mathcal{L}_{out}$ 
14:      Update parameters in the explainer with
        backpropagation
15:    end if
16:   end for
17:   epoch  $\leftarrow$  epoch + 1
18: end while

```

---

Building on our analysis in Section V-A, we construct the proxy graph by integrating the explanation subgraph  $G_{exp}$  with the perturbation of its non-explanation subgraph  $G^\Delta = (\mathcal{V}, \mathcal{E}^\Delta, \mathbf{X})$ . Here, the edge set  $\mathcal{E}^\Delta$  is defined as the differential set  $\mathcal{E} - \mathcal{E}_{exp}$ , and the adjacency matrix  $\mathbf{A}^\Delta$  is derived through  $\mathbf{A} - \mathbf{A}_{exp}$ . To further enhance the representation of proxy graphs, ProxyExplainer replaces random edge removal with a learnable representation, introducing a dual-structured mechanism comprising two distinct Graph Autoencoders (GAE) [3]. More specifically, we first include an encoder to learn a latent matrix  $\mathbf{Z}_{exp}$  according to  $\mathbf{A}_{exp}$  and  $\mathbf{X}_{exp}$ , and a decoder network recovers  $\tilde{\mathbf{A}}_{exp}$  based on  $\mathbf{Z}_{exp}$ . Formally, we have:

$$\mathbf{Z}_{exp} = \text{ENC}_1(\mathbf{A}_{exp}, \mathbf{X}_{exp}), \quad \tilde{\mathbf{A}}_{exp} = \text{DEC}(\mathbf{Z}_{exp}), \quad (13)$$

where  $\text{ENC}_1$  and  $\text{DEC}$  are the encoder network and decoder network. ProxyExplainer is flexible to the choices of these two networks.  $\tilde{\mathbf{A}}_{exp} \in \mathbb{R}^{n \times n}$  is the reconstructed adjacency matrix.

To introduce perturbations into the non-explanation segment, ProxyExplainer employs a VGAE, a generative model adept at creating varied yet structurally coherent graph data. This capability is pivotal to generate nuanced variations of the non-explanation subgraph. The VGAE operates by first encoding the non-explanation subgraph  $G^\Delta$  into a latent probabilistic space, characterized by Gaussian distributions. This process is articulated as:

$$\boldsymbol{\mu}^\Delta = \text{ENC}_1(\mathbf{A}^\Delta, \mathbf{X}), \quad \boldsymbol{\sigma}^\Delta = \text{ENC}_2(\mathbf{A}^\Delta, \mathbf{X}), \quad (14)$$

where  $\text{ENC}_1$  and  $\text{ENC}_2$  are encoder networks that learn the mean  $\boldsymbol{\mu}^\Delta$  and variance  $\boldsymbol{\sigma}^\Delta$  of the Gaussian distributions, respectively. Following this, the latent representations  $\mathbf{Z}^\Delta$  are sampled from these distributions, ensuring that each generated instance is a unique variation of the original. The decoder network then

TABLE V  
EXPLANATION FIDELITY ON LARGE DATASETS AND SOCIAL NETWORK DATASETS

|              | D&D                          |                                | REDDIT-BINARY                |                                | OGBG-MolHIV                  |                                | IMDB-BINARY                  |                                |
|--------------|------------------------------|--------------------------------|------------------------------|--------------------------------|------------------------------|--------------------------------|------------------------------|--------------------------------|
|              | $Fid_{\alpha_1, +} \uparrow$ | $Fid_{\alpha_2, -} \downarrow$ | $Fid_{\alpha_1, +} \uparrow$ | $Fid_{\alpha_2, -} \downarrow$ | $Fid_{\alpha_1, +} \uparrow$ | $Fid_{\alpha_2, -} \downarrow$ | $Fid_{\alpha_1, +} \uparrow$ | $Fid_{\alpha_2, -} \downarrow$ |
| GradCAM      | 0.020±0.000                  | 0.030±0.000                    | 0.065±0.000                  | 0.053±0.000                    | 0.017±0.000                  | 0.026±0.000                    | 0.006±0.000                  | 0.009±0.000                    |
| GNNExp.      | 0.017±0.001                  | 0.023±0.001                    | 0.061±0.006                  | 0.057±0.004                    | 0.022±0.002                  | 0.028±0.002                    | 0.003±0.002                  | 0.015±0.001                    |
| PGExp.       | 0.016±0.001                  | 0.013±0.001                    | 0.070±0.003                  | 0.031±0.002                    | 0.028±0.005                  | 0.023±0.005                    | 0.003±0.001                  | 0.014±0.001                    |
| ReFine       | 0.020±0.000                  | 0.022±0.000                    | 0.062±0.004                  | 0.047±0.003                    | 0.030±0.000                  | 0.027±0.001                    | 0.006±0.001                  | 0.010±0.001                    |
| MixupExp.    | 0.018±0.002                  | 0.014±0.002                    | 0.067±0.004                  | 0.032±0.002                    | 0.026±0.002                  | 0.023±0.001                    | 0.003±0.001                  | 0.009±0.001                    |
| EdgedropExp. | 0.019±0.002                  | 0.012±0.002                    | 0.072±0.003                  | 0.030±0.002                    | 0.029±0.005                  | 0.020±0.005                    | 0.004±0.001                  | 0.008±0.001                    |
| ProxyExp.    | 0.022±0.002                  | 0.010±0.002                    | 0.074±0.006                  | 0.031±0.002                    | 0.030±0.003                  | 0.021±0.002                    | 0.007±0.002                  | 0.006±0.002                    |

TABLE VI  
EXPLANATION ACCURACY (AUC-ROC) FOR NODE CLASSIFICATION WITH  
GROUND-TRUTH ANNOTATIONS

|                  | Tree-Cycles | Tree-Grid   |
|------------------|-------------|-------------|
| GradCAM          | 0.737±0.000 | 0.590±0.000 |
| GNNExpainer      | 0.762±0.009 | 0.672±0.002 |
| PGExpainer       | 0.760±0.012 | 0.675±0.008 |
| MixupExpainer    | 0.771±0.008 | 0.700±0.001 |
| EdgedropExpainer | 0.774±0.018 | 0.693±0.006 |
| ProxyExpainer    | 0.783±0.009 | 0.711±0.002 |

reconstructs the perturbed non-explanation subgraph from these sampled latent representations:

$$\mathbf{Z}^\Delta \sim \mathcal{N}(\boldsymbol{\mu}^\Delta, \text{diag}(\boldsymbol{\sigma}^\Delta)^2), \quad \tilde{\mathbf{A}}^\Delta = \text{DEC}(\mathbf{Z}^\Delta), \quad (15)$$

where  $\tilde{\mathbf{A}}^\Delta \in \mathbb{R}^{n \times n}$  represents the adjacency matrix of the perturbed non-explanation subgraph. This novel use of VGAE facilitates the generation of diverse yet representative perturbations, which are crucial for enhancing the interpretability of explainers in ProxyExpainer. The adjacency matrix of a proxy graph is then obtained by

$$\tilde{\mathbf{A}} = \tilde{\mathbf{A}}_{exp} + \tilde{\mathbf{A}}^\Delta. \quad (16)$$

*Loss function:* To train the proxy graph generator, we introduce a standard Gaussian distribution as the prior for the latent space, i.e.,  $\mathbf{Z} \sim \mathcal{N}(0, \mathcal{I})$ . The loss function is

$$\mathcal{L}_{gen} = \mathcal{L}_{in} + \lambda \mathcal{L}_{KL}, \quad (17)$$

where  $\mathcal{L}_{in}$  is defined in (11), and  $\mathcal{L}_{KL}$  is the KL divergence between the latent distribution  $q(\mathbf{Z}^\Delta | G^\Delta)$  and the Gaussian prior. Parameter  $\lambda$  controls the regularization strength.

*Alternate Training:* To solve the bi-level optimization in (8), we employ alternating gradient updates. The inner optimization updates the proxy graph generator through  $\mathcal{L}_{gen}$ , while the outer optimization updates the explainer through  $\mathcal{L}_{out}$  with the generator fixed. Following existing works [53], we train the proxy graph generator  $M$  times for each explainer update, where  $M$  is determined by grid search. Algorithm 2 provides the detailed procedure.

## D. Complexity Analysis

In this part, we analyze the time and space complexity of the proposed methods w.r.t the number of nodes  $n$  and edges  $|\mathcal{E}|$ . We treat hidden dimensions as constants since they are independent of the input graph size.

A key practical advantage of our framework is that *the proxy graph generator introduces no overhead during inference*. The proxy graphs are constructed solely to provide robust training signals for the explainer by mitigating distribution shifts. Once training is complete, only the learned explainer  $\Psi(\cdot)$  is retained for generating explanations on new graphs. At inference time, given a new graph  $G$ , the explainer directly outputs the explanation subgraph  $G_{exp} = \Psi(G)$  without invoking the proxy graph generator. Therefore, the inference complexity of both EdgedropExpainer and ProxyExpainer is identical to that of PGExpainer, i.e.,  $O(|\mathcal{E}|)$ , making our methods readily deployable in latency-sensitive applications. We next provide detailed training time complex analysis and comparison.

*EdgedropExpainer:* The main computational cost of EdgedropExpainer consists of the explainer forward pass and the edge sampling operation. For the explainer, computing edge probabilities via the MLP  $g_\psi$  requires  $O(|\mathcal{E}|)$  time. The edge sampling operation on  $G - G_{exp}$  is also  $O(|\mathcal{E}|)$ . Thus, the overall time complexity per iteration is  $O(|\mathcal{E}|)$ . The space complexity is  $O(|\mathcal{E}| + n)$  for storing the adjacency matrix and node embeddings.

*ProxyExpainer:* In addition to the explainer, ProxyExpainer involves the proxy graph generator consisting of a GAE and a VGAE. For the GAE encoding  $G_{exp}$ , the time complexity is  $O(|\mathcal{E}_{exp}| + n)$  for GNN message passing and feature transformation. The VGAE for  $G^\Delta$  has similar complexity  $O(|\mathcal{E}^\Delta| + n)$ . The decoder reconstructs the adjacency matrix with complexity  $O(n^2)$  via inner product operations. Therefore, the overall time complexity per iteration is  $O(|\mathcal{E}| + n^2)$ . For sparse graphs where  $|\mathcal{E}| = O(n)$ , this simplifies to  $O(n^2)$ . The space complexity is  $O(n^2)$  for storing the reconstructed adjacency matrix.

*Comparison with Baselines:* Table II summarizes the complexity comparison. PGExpainer has the lowest complexity as it only requires the explainer forward pass. MixupExpainer incurs additional cost for sampling and combining graphs from different instances, resulting in  $O(|\mathcal{E}| + n^2)$  complexity. Our EdgedropExpainer achieves comparable complexity to PGExpainer, while ProxyExpainer introduces moderate overhead from the generative model but remains efficient for graphs of practical sizes, as validated empirically in Fig. 11.

## VII. EXPERIMENTS

We present empirical results that illustrate the effectiveness of our proposed method. These experiments are mainly designed to explore the following research questions:

- *RQ1*: Can the proposed framework outperform other baselines in identifying explanations for GNNs?
- *RQ2*: Is the distribution shift severe in explanation subgraphs? Can the proposed approach alleviate it?
- *RQ3*: How does each component of ProxyExplainer impact the overall performance in generating explanations?

### A. Experimental Setup

1) *Datasets*: We use seven benchmark real-world datasets and two synthetic datasets to evaluate the performance. The statistics of all datasets are summarized in Table I.

**MUTAG [21]**: MUTAG dataset includes 4,337 molecular graphs, each of them is classified into two groups based on its mutagenic effect on the Gram-negative bacterium *S. Typhimurium*. This classification comes from several specific virulence gland groups with mutagenicity in molecular mapping by Kazius et al. [21].

**Benzene [36]**: Benzene consists 12,000 molecular graphs from the ZINC15 [39] database, which can be classified into two classes. The main goal is to determine if a Benzene ring exists in each molecule. In cases where multiple Benzene rings are present, each Benzene ring is treated as a distinct explanation.

**Alkane-Carbonyl [36]**: The Alkane-Carbonyl dataset consists of a total of 4,326 molecule graphs that are categorized into two different classes. Positive samples refer to molecules that have both alkane and carbonyl functional groups. The ground-truth explanation includes alkane and carbonyl functional groups in a given molecule.

**Fluoride-Carbonyl [36]**: The Fluoride-Carbonyl dataset has 8,671 molecular graphs. The ground-truth explanation is based on the particular combination of fluoride atoms and carbonyl functional groups present in each molecule.

**BA-2motifs [28]**: The BA-2motifs dataset consists 1,000 synthetic graphs, each graph is derived from a basic Barabasi-Albert (BA) model. The dataset is divided into two categories: one part of the graphs add patterns that mimic the structure of a house, and the remaining integrate five-node cyclic patterns. The classification depends on the specific patterns.

**BA-3motifs [7]**: BA-3motifs is an extended dataset inspired by the BA-2motifs and contains 3,000 synthetic graphs. Each base graph is accompanied by one of three different patterns: house, cycle, or grid.

**D&D [14]**: This benchmark contains 1,178 graphs derived from protein structures, labeled as enzymes or non-enzymes. Each node in the graph represents an amino acid, and an edge is formed between two amino acids whenever their spatial distance is within 6 Angstroms, capturing local geometric interactions in the protein.

**OGBG-MolHIV [30]**: The dataset includes 41,127 graphs, each representing a compound evaluated for its capability to inhibit HIV replication. These graphs are assigned to one of two classes: active molecules, indicating antiviral effectiveness; inactive, indicating no significant inhibitory behavior.

**IMDB-BINARY [46]**. This dataset consists of 1,000 ego-networks constructed from IMDB cast information. Nodes correspond to actors or actresses, and an edge is added when two individuals appear together in a movie.

**REDDIT-BINARY [46]**. This dataset consists of 2,000 graphs collected from Reddit discussion threads. Each graph represents a discussion thread, where nodes correspond to users and edges indicate reply interactions between users. The graphs are labeled according to whether the thread originates from a question/answer-based or discussion-based community.

2) *Baselines*: To evaluate the proposed methods, we well-train the GCN model to ensure good performance in graph classification tasks. For a comprehensive comparison, we incorporate various post-hoc explanation methods.

**GradCAM [33]** utilizes gradients as a weighting mechanism to merge various feature maps. It operates on heuristic assumptions and cannot elucidate node classification models.

**GNNExplainer [47]** learns soft masks for edges and node features, and aims to elucidate predictions through mask optimization. GNNExplainer integrates these masks with the original graph through element-wise multiplications. The masks are optimized by enhancing the mutual information between the original graph and the modified graph prediction results.

**PGExplainer [28]** extends the idea of GNNExplainer by assuming that the graph is a random Gilbert graph. PGExplainer generates each edge embedding by combining the embeddings of its constituent nodes, then uses these edge embeddings to determine a Bernoulli distribution to indicate whether to mask an edge or not, and utilizes a Gumbel-Softmax approach to model the Bernoulli distribution for end-to-end training.

**ReFine [43]** identifies the edge probabilities for the entire category by maximizing the mutual information and contrastive loss between categories. In fine-tuning, it uses the edge probabilities from the previous stage to sample edges, and find explanations that maximize mutual information for instances.

**MixupExplainer [51]** combines original explanation subgraphs with randomly sampled, label-independent base graphs in a non-parametric way to mitigate the common OOD issue which found in previous methods.

3) *Evaluation Metrics*: Quantitative evaluation of explanations can be categorized into two paradigms: 1) *accuracy*, which measures alignment with ground-truth annotations when available; and 2) *fidelity* (also known as faithfulness), which measures prediction consistency between the original graph and explanation subgraph. We adopt both metrics depending on data availability.

*Accuracy (Comparison to Ground Truth)*: When ground-truth explanations are available, we formulate the explanation task as binary edge classification: edges within ground-truth explanation subgraphs are positive, and all others are negative. The importance weights from explanation methods serve as prediction scores, and we use AUC-ROC for quantitative assessment following prior work [28], [47].

This evaluation is valid under the standard assumption that the to-be-explained GNN is well-trained and accurate. When the model correctly learns to rely on ground-truth explanation patterns for prediction, a faithful explanation that accurately reflects the model's decision process should align with these patterns. Thus, accuracy with respect to ground truth serves as a proxy for faithfulness in this setting.

*Robust Fidelity (Faithfulness to Original Prediction)*: When ground-truth explanations are unavailable, we evaluate how well

TABLE VII

MMD RESULTS BETWEEN THE GROUND TRUTH EXPLANATIONS AND ORIGINAL GRAPHS (GT); PGEXPLAINER EXPLANATIONS AND ORIGINAL GRAPHS (PGE); MIXUPEXPLAINER EXPLANATIONS AND ORIGINAL GRAPHS (MIXUP); PROXY GRAPHS IN OUR METHODS AND ORIGINAL GRAPHS (EDGEDROP AND PROXY)

|        | MUTAG               |              |       |              |              | Fluoride-Carbonyl |              |              |              |              | BA-2motifs   |       |              |              |              |
|--------|---------------------|--------------|-------|--------------|--------------|-------------------|--------------|--------------|--------------|--------------|--------------|-------|--------------|--------------|--------------|
|        | Gaussian EMD Kernel |              |       |              |              |                   |              |              |              |              |              |       |              |              |              |
| Metric | GT                  | PGE          | Mixup | Edgedrop     | Proxy        | GT                | PGE          | Mixup        | Edgedrop     | Proxy        | GT           | PGE   | Mixup        | Edgedrop     | Proxy        |
| Deg.   | 0.614               | 0.468        | 0.207 | <u>0.162</u> | <b>0.123</b> | 0.638             | 0.488        | <b>0.137</b> | 0.206        | <u>0.196</u> | 0.759        | 0.496 | <b>0.052</b> | <u>0.054</u> | 0.060        |
| Clus.  | <b>0.003</b>        | <b>0.003</b> | 0.052 | 0.018        | <u>0.009</u> | <b>0.012</b>      | <b>0.012</b> | 0.347        | <u>0.095</u> | <b>0.012</b> | <u>0.447</u> | 0.463 | 0.642        | <b>0.369</b> | 0.584        |
| Spec.  | 0.414               | 0.341        | 0.190 | <b>0.179</b> | <u>0.186</u> | 0.351             | 0.315        | <u>0.063</u> | <b>0.056</b> | 0.100        | 0.245        | 0.256 | <u>0.073</u> | <b>0.046</b> | 0.091        |
| Sum.   | 1.031               | 0.812        | 0.449 | <u>0.359</u> | <b>0.318</b> | 1.001             | 0.815        | 0.547        | <u>0.357</u> | <b>0.308</b> | 1.451        | 1.215 | 0.767        | <b>0.469</b> | <u>0.735</u> |
|        | Linear Kernel       |              |       |              |              |                   |              |              |              |              |              |       |              |              |              |
| Metric | GT                  | PGE          | Mixup | Edgedrop     | Proxy        | GT                | PGE          | Mixup        | Edgedrop     | Proxy        | GT           | PGE   | Mixup        | Edgedrop     | Proxy        |
| Deg.   | 0.358               | 0.246        | 0.105 | <u>0.103</u> | <b>0.084</b> | 0.286             | 0.232        | <u>0.058</u> | 0.123        | <b>0.044</b> | 0.509        | 0.321 | <b>0.002</b> | <u>0.003</u> | 0.005        |
| Clus.  | <b>0.012</b>        | <b>0.012</b> | 0.018 | 0.016        | <u>0.015</u> | <b>0.008</b>      | <u>0.012</u> | 0.171        | 0.022        | <b>0.008</b> | <u>0.203</u> | 0.212 | 0.451        | <b>0.102</b> | 0.304        |
| Spec.  | 0.025               | 0.028        | 0.011 | <u>0.007</u> | <b>0.005</b> | 0.010             | 0.018        | <u>0.006</u> | <b>0.005</b> | 0.008        | 0.090        | 0.097 | <b>0.001</b> | <b>0.001</b> | <u>0.002</u> |
| Sum.   | 0.395               | 0.286        | 0.134 | <u>0.126</u> | <b>0.104</b> | 0.304             | 0.262        | 0.235        | <u>0.163</u> | <b>0.060</b> | 0.802        | 0.630 | 0.454        | <b>0.106</b> | <u>0.311</u> |

the explanation reflects the model’s predictive behavior. Standard fidelity metrics measure changes in classifier output when the explanation subgraph is retained or removed, but they become unreliable because of the distribution shift problem [48]. We therefore adopt *Robust Fidelity* [53], which uses stochastic edge sampling functions  $E_\alpha$  with probability  $\alpha \in [0, 1]$ :

$$Fid_{\alpha_1,+} \triangleq f(G)_y - \mathbb{E}f(G - E_{\alpha_1}(G_{exp}))_y,$$

$$Fid_{\alpha_2,-} \triangleq f(G)_y - \mathbb{E}f(G_{exp} + E_{\alpha_2}(G - G_{exp}))_y, \quad (18)$$

where setting  $\alpha_1 < 1$  and  $\alpha_2 > 0$  mitigates OOD issues. We use the default settings ( $\alpha_1 = 0.1$ ,  $\alpha_2 = 0.9$ ).

4) *Settings*: We follow the experimental setting in previous works [28], [36], [47] to train a Graph Convolutional Network (GCN) model [23] with three layers. We use the Adam optimizer [22] with a weight decay  $5e - 4$ .

### B. Quantitative Evaluation (RQ1)

To answer RQ1, we compare the proposed methods, EdgedropExplainer and ProxyExplainer, to other baselines. Each experiment was conducted 10 times using random seeds, and the average AUC scores as well as standard deviations are presented in Table III. The results demonstrate that ProxyExplainer provides the most accurate explanations across all datasets. Specifically, it improves the AUC scores by an average of 6.9% on real-world datasets except Alkane-Carbonyl, and 7.5% on synthetic datasets over the leading baselines. Comparisons with baseline methods highlight the advantages of our proposed explanation framework. Besides, ProxyExplainer captures underlying explanation factors consistently across diverse datasets. For instance, MixupExplainer exhibits proficiency on the synthetic BA-2motifs dataset but performs poorly on the real-world Benzene dataset. The reason is that MixupExplainer relies on the independence assumption between explanation and non-explanation subgraphs, which may not hold in real-world datasets. In contrast, ProxyExplainer consistently demonstrates high performance across different datasets, showcasing its robustness and adaptability. These results also show that ProxyExplainer outperforms on large-scale datasets (D&D, OGBG-MolHIV) and on the social network dataset IMDB-BINARY, demonstrating that the proposed framework scales effectively

beyond small datasets and remains effective in the social network domain.

Moreover, Tables IV and V demonstrate that ProxyExplainer outperforms most baseline methods in both  $Fid_{\alpha_1,+}$  and  $Fid_{\alpha_2,-}$ . For  $Fid_{\alpha_1,+}$ , ProxyExplainer achieves the highest or near-highest scores across multiple datasets, showing a clear advantage, particularly on MUTAG and BA-2motifs. For  $Fid_{\alpha_2,-}$ , it consistently achieves the lowest scores, indicating a strong performance in preserving negative fidelity. These results highlight the superior fidelity of ProxyExplainer in model interpretation.

Finally, we conducted additional experiments on node classification using the Tree-Cycles and Tree-Grid datasets [47]. As shown in Table VI, our framework achieves high-quality explanations on the node classification task. These results further demonstrate its generality beyond graph-level tasks.

### C. Alleviating Distribution Shifts (RQ2)

1) *Quantitative Analysis*: In this section, we assess our methods’ ability to generate in-distribution proxy graphs.

*Gaussian kernel MMD*: Due to the intractability of direct computation, we follow the previous work [7] to utilize Maximum Mean Discrepancy (MMD) between the generated proxy graphs and original graphs, based on their distributions of multiple graph statistics, including degree distributions, clustering coefficients, and spectrum distributions. Specifically, we use Gaussian Earth Mover’s Distance (EMD) kernel when computing MMDs. Smaller MMD values indicate higher similarity between graph distributions. For comparison, we also include the ground-truth explanations and those generated by PGExplainer and MixupExplainer.

The results are shown in Table VII. “GT” denotes the MMDs between the ground-truth explanations and original graphs. “PGE” represents the MMDs between explanations generated by PGExplainer and original graphs. “Mixup” combines the explanation subgraph with a randomly sampled, label-independent base graph. “Edgedrop” and “Proxy” denote the MMDs between proxy graphs in our methods and original graphs. We have the following observations. First, the MMDs between ground truth and original graphs are usually large, verifying our motivation that a model trained on original graphs may not make correct

predictions on the OOD explanation subgraphs. Second, the explanations generated by a representative method, PGExplainer, are often OOD from original graphs, indicating that the original GIB-based objective function may be sub-optimal. Third, we observe that proxy graphs often produce lower MMD values than MixupExplainer. This shows that conditioning on the original graph better preserves its structural statistics than mixing with independent graphs. Finally, in most cases, proxy graphs generated by our methods have smaller MMDs, demonstrating their in-distribution property.

**Linear kernel MMD:** We replace the Gaussian EMD kernel to a linear kernel to calculate MMD. As shown in Table VII, it produces the same relative ranking across methods, indicating that our MMD evaluation is robust to kernel choice.

2) **Qualitative Analysis:** In addition, we evaluate our method using classification accuracy and graph embedding distribution. Specifically, we assess the GCN classification accuracy on the original graphs, ground-truth explanation graphs, and proxy graphs. The graph embeddings of these three types of graphs are also visualized.

Fig. 4 presents the graph embedding distributions of different methods in BA-2motifs. A pronounced distribution shift is observed between ground-truth explanation graphs and original graphs, reinforcing the challenge of OOD explanations. However, proxy graphs generated by different methods closely align with the original embedding distribution, demonstrating their effectiveness in addressing the OOD problem. Fig. 5 illustrates that the trained GCN achieves high classification accuracy on original graphs; however, its performance drops significantly on ground-truth explanation graphs, highlighting the OOD problem. In contrast, the higher accuracy of proxy graphs relative to ground-truth explanation graphs suggests that our method effectively mitigates distribution shifts.

#### D. Ablation Studies (RQ3)

In this section, we conduct ablation studies to investigate the roles of different components. Specifically, we consider the following variants of ProxyExplainer: 1) w/o  $G^\Delta$ : in this variant, we remove the non-explanation subgraph generator (VGAE), which is the bottom half of Fig. 2; 2) w/o  $\mathcal{L}_{\text{size}}$ : in this variant, we remove the KL divergence term from the training loss in ProxyExplainer; 3) w/o  $\mathcal{L}_{\text{in}}$ : in this variant, we remove the distribution loss from ProxyExplainer. The results of the ablation study on MUTAG and BA-2motifs are reported in Fig. 6.

Fig. 6 illustrates a notable performance drop for all variants, indicating that each component contributes positively to the effectiveness of ProxyExplainer. Especially, in the real-world dataset MUTAG, the absence of the in-distribution constraint (w/o  $\mathcal{L}_{\text{in}}$ ) results in significantly worse performance compared to ProxyExplainer, indicating the crucial role of the in-distribution proxy graphs in our framework.

#### E. Parameter Sensitivity Analysis

In this section, we analyze the influence of parameters, including  $\lambda$ , which controls the KL divergence during the proxy graph generation process, and the dimension  $D$  of the node

latent embeddings. We vary  $\lambda$  from 0.01 to 1.0. For  $D$ , we test values from {32, 64, 128, 256, 512, 1024}. Throughout the experiments, we adjust only one parameter at a time while keeping the remaining parameters fixed at their respective optimal values. Fig. 7 shows the performance of our ProxyExplainer with respect to  $\lambda$  on MUTAG and BA-2motifs datasets. We observe that ProxyExplainer consistently outperforms the best baseline for  $\lambda \in [0.25, 1.0]$ , indicating that ProxyExplainer is stable. Fig. 8 presents how the dimension of the node latent embedding affects performance in ProxyExplainer. The results show that ProxyExplainer achieves optimal performance at  $D = 512$ .

#### F. Case Studies

In this section, we conduct case studies to qualitatively compare the performance of ProxyExplainer against other methods. We use examples from Benzene and BA-2motifs. We present visualization results in Figs. 9 and 10. Explanations are highlighted with bold black and bold orange edges, respectively. The results show that ProxyExplainer stands out by generating more compelling explanations compared to the baselines. Specifically, ProxyExplainer maintains clarity without introducing irrelevant edges and produces more concise results than the alternative methods. The visualized performance underscores ProxyExplainer's ability in providing meaningful and focused subgraph explanations.

#### G. Efficiency Analysis

Fig. 11 illustrates the per-epoch training time of explanation methods. EdgedropExplainer achieves the lowest training time. We observe that ProxyExplainer is empirically faster than MixupExplainer in our experiments. This difference is likely due to implementation factors, as MixupExplainer requires sampling and combining subgraphs across different graph instances, which introduces additional overhead in practice. ProxyExplainer is moderately slower than EdgedropExplainer due to the variational encoding step but remains competitive overall. These results confirm that our frameworks are computationally practical for real-world applications. In addition, Fig. 12 shows the explainer training loss curves of MixupExplainer and ProxyExplainer (outer loss). MixupExplainer converges faster but plateaus at a higher loss value, whereas ProxyExplainer converges more gradually and reaches a lower loss value, indicating more stable optimization process.

### VIII. CONCLUSION

In this paper, we explore the challenge of explaining GNNs in graph classification tasks. We emphasize the difficulty of directly using explanation subgraphs, as distribution shifts in the original training data hinder accurate label prediction. To address this issue, we use non-parametric and parametric methods to generate proxy graphs for explanation subgraphs. These proxy graphs align with the training data distribution while preserving essential explanatory factors, increasing the reliability of label approximation for explanation subgraphs. Additionally, we investigate the limitations of traditional fidelity metrics  $Fid_+$  and  $Fid_-$

in measuring explanation fidelity, highlighting that distribution shifts between original and explanation subgraphs can lead to unreliability. To address this, We introduce a set of evaluation metrics, which utilize a sampling technique to mitigate such distribution shift issues and avoid evaluating out-of-distribution inputs during evaluation. Empirical evaluations across different datasets validate the effectiveness of our method, demonstrating improved accuracy in GNN explanations. While the explainer converged reliably in our experiments, the bi-level alternating optimization may lead to slower stabilization compared to single-stage training schemes. Moreover, our frameworks involve additional processing, such as variational encoding in the parametric approach, which increases training time compared to lighter-weight explainers. Exploring more computationally efficient or convergence-guaranteed optimization strategies represents a valuable direction for future work.

### ACKNOWLEDGMENT

The views and conclusions contained in this paper are those of the authors and should not be interpreted as representing any funding agencies.

### REFERENCES

- [1] N. Akhtar, M. Amir, and A. Khan Jalwana, "Rethinking interpretation: Input-agnostic saliency mapping of deep visual classifiers," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 178–186.
- [2] K. Amara, M. El-Assady, and R. Ying, "GInX-Eval: Towards in-distribution evaluation of graph neural network explanations," 2023, *arXiv:2309.16223*.
- [3] D. Bank, N. Koenigstein, and R. Giryes, "Autoencoders," in *Machine Learning for Data Science Handbook: Data Mining and Knowledge Discovery Handbook*. Berlin, Germany: Springer Nature, 2023, pp. 353–374.
- [4] M. Bianchi, Antonio De Santis, A. Tocchetti, and M. Brambilla, "Interpretable network visualizations: A human-in-the-loop approach for post-hoc explainability of CNN-based image classification," in *Proc. 33rd Int. Joint Conf. Artif. Intell.*, 2024, pp. 3715–3723.
- [5] C.-H. Chang, E. Creager, A. Goldenberg, and D. Duvenaud, "Explaining image classifiers by counterfactual generation," in *Proc. Int. Conf. Learn. Representations*, 2019.
- [6] C. Chen, O. Li, D. Tao, A. Barnett, C. Rudin, and J. K. Su, "This looks like that: Deep learning for interpretable image recognition," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 8928–8939.
- [7] J. Chen, S. Wu, A. Gupta, and Z. Ying, "D4explainer: In-distribution explanations of graph neural network via discrete denoising diffusion," in *Proc. 37th Conf. Neural Inf. Process. Syst.*, 2023, pp. 78964–78986.
- [8] R. Chen et al., "Interpreting object-level foundation models via visual precision search," in *Proc. Comput. Vis. Pattern Recognit. Conf.*, 2025, pp. 30042–30052.
- [9] R. Chen et al., "Less is more: Efficient black-box attribution via minimal interpretable subset selection," 2025, *arXiv:2504.00470*.
- [10] R. Chen, H. Zhang, S. Liang, J. Li, and X. Cao, "Less is more: Fewer interpretable region via submodular subset selection," in *Proc. 12th Int. Conf. Learn. Representations*, 2024.
- [11] J. Chen, K. Amara, J. Yu., and R. Ying, "Generative explanations for graph neural network: Methods and evaluations," *IEEE Data Eng. Bull.*, vol. 47, no. 2, pp. 64–79, 2023.
- [12] T. M. Cover. *Elements of Information Theory*. Hoboken, NJ, USA: Wiley, 1999.
- [13] A. K. Debnath, R. L. L. de Compadre, G. Debnath, A. J. Shusterman, and C. Hansch, "Structure-activity relationship of mutagenic aromatic and heteroaromatic nitro compounds. Correlation with molecular orbital energies and hydrophobicity," *J. Med. Chem.*, vol. 34, no. 2, pp. 786–797, 1991.
- [14] Paul D. Dobson and A. J. Doig, "Distinguishing enzyme structures from non-enzymes without alignments," *J. Mol. Biol.*, vol. 330, no. 4, pp. 771–783, 2003.
- [15] J. Fang et al., "Evaluating post-hoc explanations for graph neural networks via robustness analysis," in *Proc. 37th Conf. Neural Inf. Process. Syst.*, 2023, pp. 72446–72463.
- [16] J. Fang et al., "On regularization for explaining graph neural networks: An information theory perspective," *IEEE Trans. Knowl. Data Eng.*, early access, Jul. 02, pp. 1–14, 2024.
- [17] J. Fang, X. Wang, A. Zhang, Z. Liu, X. He, and T. S. Chua, "Cooperative explanations of graph neural networks," in *Proc. 16th ACM Int. Conf. Web Search Data Mining*, 2023, pp. 616–624.
- [18] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, 1024–1034.
- [19] R. Huang, F. Shirani, and D. Luo, "Factorized explainer for graph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 12626–12634.
- [20] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with gumbel-softmax," in *Proc. Int. Conf. Learn. Representations*, 2017.
- [21] J. Kazius, R. McGuire, and R. Bursi, "Derivation and validation of toxicophores for mutagenicity prediction," *J. Med. Chem.*, vol. 48, no. 1, pp. 312–320, 2005.
- [22] P. Diederik Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Representations*, 2015.
- [23] Thomas N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2017.
- [24] Pang Wei Koh et al., "Concept bottleneck models," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 5338–5348.
- [25] W. Li, Y. Li, Z. Li, J. Hao, and Y. Pang, "DAG matters! GFlowNets enhanced explainer for graph neural networks," in *Proc. The 11th Int. Conf. Learn. Representations*, 2023.
- [26] Y. Li, J. Zhou, S. Verma, and F. Chen, "A survey of explainable graph neural networks: Taxonomy and evaluation metrics," 2022, *arXiv:2207.12599*.
- [27] Y. Liao, Y. Gao, and W. Zhang, "Feature activation map: Visual explanation of deep learning models for image classification," 2023, *arXiv:2307.05017*.
- [28] D. Luo et al., "Parameterized explainer for graph neural network," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 19620–19631.
- [29] D. Luo et al., "Towards inductive and efficient explanations for graph neural networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 8, pp. 5245–5259, Aug. 2024.
- [30] Kha-Dinh Luong and A. K. Singh, "Fragment-based pretraining and fine-tuning on molecular graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, vol. 36, pp. 17584–17601.
- [31] C. J. Maddison, A. Mnih, and Y. W. Teh, "The concrete distribution: A continuous relaxation of discrete random variables," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–20.
- [32] S. Miao, M. Liu, and P. Li, "Interpretable and generalizable graph learning via stochastic attention mechanism," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 15524–15543.
- [33] P. E. Pope, S. Kolouri, M. Rostami, C. E. Martin, and H. Hoffmann, "Explainability methods for graph convolutional neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 10772–10781.
- [34] J. Qiu, J. Tang, H. Ma, Y. Dong, K. Wang, and J. Tang, "DeepInf: Social influence prediction with deep learning," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 2110–2119.
- [35] L. Qiu et al., "Generating perturbation-based explanations with robustness to out-of-distribution data," in *Proc. ACM Web Conf.*, 2022, pp. 3594–3605.
- [36] B. Sanchez-Lengeling et al., "Evaluating attribution for graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 5898–5910.
- [37] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2009.
- [38] P. Schwab and W. Karlen, "CXPlain: Causal explanations for model interpretation under uncertainty," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 10188–10198.
- [39] T. Sterling and John J. Irwin, "ZINC 15 - ligand discovery for everyone," *J. Chem. Inf. Model.*, vol. 55, no. 11, pp. 2324–2337, 2015.
- [40] J. Tan et al., "Learning and evaluating graph neural network explanations based on counterfactual and factual reasoning," in *Proc. ACM web Conf.*, 2022, pp. 1018–1027.

- [41] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018.
- [42] H. Wang, S. Tan, and H. Wang, "Probabilistic conceptual explainers: Trustworthy conceptual explanations for vision foundation models," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024, pp. 51502–51522.
- [43] X. Wang, Y. Wu, and A. Zhang, "Xiangnan he, and tat-seng chua. towards multi-grained explainability for graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 18446–18458.
- [44] B. Wu et al., "A survey of trustworthy graph learning: Reliability, explainability, and privacy protection," 2022, *arXiv:2205.10014*.
- [45] Z. Wu et al., "Moleculenet: A benchmark for molecular machine learning," *Chem. Sci.*, vol. 9, no. 2, pp. 513–530, 2018.
- [46] P. Yanardag and S. V. N. Vishwanathan, "Deep graph kernels," in *Proc. 21th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2015, pp. 1365–1374.
- [47] Z. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "GNNEExplainer: Generating explanations for graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, vol. 32, pp. 9240–9251.
- [48] H. Yuan, H., S. Yu Gui, and S. Ji, "Explainability in graph neural networks: A taxonomic survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 5, pp. 5782–5799, May 2023.
- [49] H. Zhang, B. Wu, X. Yuan, S. Pan, H. Tong, and J. Pei, "Trustworthy graph neural networks: Aspects, methods, and trends," *Proc. IEEE*, vol. 112, no. 2, pp. 97–139, Feb. 2024.
- [50] J. Zhang et al., "RegExplainer: Generating explanations for graph neural networks in regression tasks," in *Proc. 2nd Learn. Graphs Conf.*, 2023, pp. 9282–79306.
- [51] J. Zhang, D. Luo, and H. Wei, "MixupExplainer: Generalizing explanations for graph neural networks with data augmentation," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2023, pp. 3286–3296.
- [52] S. Zhang, Y. Liu, N. Shah, and Y. Sun, "Gstarx: Explaining graph neural networks with structure-aware cooperative games," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 19810–19823.
- [53] X. Zheng, F. et al., "Towards robust fidelity for evaluating explainability of graph neural networks," in *Proc. 12th Int. Conf. Learn. Representations*, 2024, pp. 12250–12275.



**Xu Zheng** received the bachelor's degree from Chongqing University, Chongqing, China, and the master's degree from University of Electronic Science and Technology of China, Chengdu, China. He is currently working toward the PhD degree with Florida International University, supervised by Prof. Dongsheng Luo. His research interests include time series learning and XAI.



**Jiaying Zhang** received the bachelor's degree from Xi'an Jiaotong University, and the PhD degree in informatics from New Jersey Institute of Technology in 2025. His research interests include explainable AI, large language models, knowledge graphs, and recommendation systems.



**Hua Wei** received the PhD degree from the College of Information Science and Technology, The Pennsylvania State University. He is currently an assistant professor with the School of Computing and Augmented Intelligence, Arizona State University. His research interests include machine learning, data mining and trustworthy AI.



**Zhuomin Chen** is currently working toward the PhD degree with the knight foundation school of computing and information sciences, Florida International University. Her interests include data mining and machine learning.



**Jingchao Ni** received the PhD degree from the College of Information Science and Technology, The Pennsylvania State University. He is currently an assistant professor with the Department of Computer Science, University of Houston. His research interests include machine learning, data mining and artificial intelligence.



**Hojat Allah Salehi** received the MSc degree in control engineering from Sharif University of Technology. He is currently working toward the PhD degree in computer science with Florida International University. His research interests include machine learning, deep learning, and artificial intelligence.



**Farhad Shirani** received the MSc and PhD degrees in electrical Engineering with the University of Michigan, Ann Arbor, and the MSc degree in applied mathematics from the University of Michigan. His research interests include explainable machine learning, privacy and security, wireless and information theory.



**Esteban Schafir** received the master's degree in cybersecurity from the Knight Foundation School of Computing and Information Sciences, Florida International University, where he is currently working toward the PhD degree in computer science. His research interests include machine learning, with particular emphasis on deep learning architectures and their applications.



**Dongsheng Luo** (Member, IEEE) received the doctoral degree with the College of Information Science and Technology, Pennsylvania State University. His research interests include trustworthy AI, data mining, and machine learning.